Python NumPy for AI & Robotics

Assignment Questions

Name: _____ Batch / ID: _____ Date: _____

Total: 100 marks Questions: 20 (5 marks each) Time: 90 minutes

- Write Python code with NumPy for every question. Start each program with `import numpy as np`.
- Where a question says "print", show the statement you would use and the output you expect.
- All scenarios are drawn from robotics and AI: sensors, joints, motors, cameras and mission logs.

Section A: Getting Started with Arrays

Q1. NumPy Intro and Getting Started [5 marks]

A robotics team stores thousands of sensor readings and finds Python lists too slow.

(a) Give two reasons why a NumPy ndarray is better than a Python list for this job.

(b) Write the statement that imports NumPy with the alias `np` and prints the installed NumPy version.

(c) What is the type of the object returned by `np.array([1, 2, 3])`?

Q2. Creating Arrays [5 marks]

An ultrasonic sensor records five distances in centimetres: 24, 31, 28, 35 and 30.

(a) Create a 1-D array named `dist` holding these values.

(b) Create a 2-D array of shape 3 x 3 named `angles` holding the joint angles of a three-link arm over three time steps (choose your own values).

(c) Print the number of dimensions (`ndim`) of both arrays.

Q3. Array Indexing [5 marks]

Consider the array below. Each row is one sensor and each column is one reading.

```
readings = np.array([[12, 15, 14],
                     [22, 25, 21],
                     [31, 30, 33],
                     [18, 17, 19]])
```

(a) Access the 3rd reading of the 2nd sensor.

(b) Use negative indexing to access the last reading of the last sensor.

(c) Change the first reading of the first sensor to 0 and print the array.

Q4. Array Slicing [5 marks]

A lidar scan returns ten distance values.

```
scan = np.array([5, 8, 9, 12, 7, 6, 15, 14, 3, 10])
```

(a) Slice elements from index 2 up to (not including) index 6.

(b) Slice every second element of the whole array.

(c) Slice the last three readings using a negative start index.

(d) For a 2-D array of shape 4 x 5, slice rows 1 to 2 and columns 0 to 2 in a single expression.

Q5. Data Types [5 marks]

Motor speeds were stored as integers: `speeds = np.array([10, 20, 30])`.

(a) Print the data type of the array.

(b) Convert the array to `float32` using `astype()` and print the new dtype.

(c) Create an array directly with the dtype `int8` and explain when a smaller data type is useful on an embedded robot.

Q6. Copy vs View [5 marks]

Calibration code changes the first value of a sensor array.

(a) Create `c = a.copy()` and `v = a.view()` from `a = np.array([1, 2, 3, 4, 5])`.

(b) Change `a[0]` to 99. Which of c and v changes, and why?

(c) Show how the `base` attribute tells you whether an array owns its data.

Section B: Shape, Iteration and Combining Arrays

Q7. Array Shape and Reshape [5 marks]

An IMU produces 12 readings per cycle.

(a) Create the 12 readings with `np.arange(1, 13)` and reshape them into 3 rows and 4 columns.

(b) Reshape the same data into a 3-D array of shape (2, 3, 2) and print its shape.

(c) Use `-1` to let NumPy work out one dimension automatically, reshaping into 4 columns.

(d) Why does reshaping 12 elements into (5, 3) raise an error?

Q8. Array Iterating [5 marks]

Use the array of joint angles below.

```
joints = np.array([[30, 45, 60],
                   [10, 20, 30]])
```

(a) Print every element using nested `for` loops.

(b) Rewrite the task with `np.nditer()`.

(c) Use `np.ndenumerate()` to print each value with its index.

Q9. Joining, Concatenating and Stacking Arrays [5 marks]

The left and right wheel speeds are `left = np.array([1, 2, 3])` and `right = np.array([4, 5, 6])`.

(a) Join them into one 1-D array using `np.concatenate()`.

(b) Stack them row by row with `np.vstack()` and column by column with `np.hstack()`. Print both shapes.

(c) Use `np.stack()` with `axis=1` and explain how it differs from concatenate.

(d) Join two 2-D arrays of shape (2, 2) along `axis=0` and then `axis=1`.

Q10. Array Split [5 marks]

A mission log holds 12 battery readings.

(a) Split the array into 3 equal parts with `np.array_split()` and print the second part.

(b) Split the same array into 5 parts. What happens to the sizes of the parts?

(c) Split a 2-D array of shape (4, 3) into two arrays along rows.

Section C: Searching, Sorting and Filtering

Q11. Array Search and Sort [5 marks]

Distances measured by a robot are `d = np.array([42, 17, 63, 8, 29, 17])`.

(a) Use `np.where()` to find the indexes where the distance equals 17.

(b) Use `np.searchsorted()` to find where 30 should be inserted so the (sorted) array stays in order.

(c) Sort the array in ascending order and confirm that the original array is unchanged.

(d) Sort a 2-D array so that each row is sorted independently.

Q12. Array Filter and Boolean Masks [5 marks]

Obstacle distances are `dist = np.array([12, 45, 7, 60, 33])`.

(a) Create a boolean list (mask) that is True for distances greater than 30.

(b) Use the mask to filter the array and print the result.

(c) Create the filter directly from a condition without building the mask separately.

Q13. Filtering with AND, OR and NOT [5 marks]

Battery levels (percent) of eight robots are given below.

```
battery = np.array([100, 85, 60, 12, 45, 95, 20, 5])
```

(a) Select robots with a battery between 20 and 80 inclusive (AND).

(b) Select robots with a battery below 15 or above 90 (OR).

(c) Select robots whose battery is NOT equal to 100 (NOT).

(d) Count how many robots satisfy part (a).

Q14. Flattening and Transposing Arrays [5 marks]

A camera frame is stored as a 2 x 3 array of pixel values.

(a) Flatten it into a 1-D array using `flatten()`.

(b) Flatten it again using `ravel()` and state one difference between the two methods.

(c) Transpose the array using `.T` and print the new shape.

(d) Explain why transposing is common when working with matrices of sensor data.

Section D: Statistics, Files and Dates

Q15. Sum, Mean and Median [5 marks]

Temperature readings from three motors (rows) over four minutes (columns) are shown below.

```
temp = np.array([[40, 42, 41, 90],
                 [38, 39, 40, 41],
                 [45, 46, 44, 45]])
```

(a) Find the total sum of all readings.

(b) Find the mean temperature of each motor (use the `axis` argument).

(c) Find the median temperature of each minute (column).

(d) Explain why the median can be better than the mean when one sensor reading is faulty.

Q16. Min, Max and Their Positions [5 marks]

Use the lidar array `scan = np.array([5.2, 3.1, 8.4, 2.7, 6.0])`.

(a) Find the minimum and maximum distance.

(b) Use `np.argmin()` to find the index of the nearest obstacle.

(c) Use `np.argmax()` to find the index of the farthest obstacle.

(d) Find the maximum of each column of a 2-D array.

Q17. File I/O with NumPy [5 marks]

A robot must save its path waypoints and load them again on the next run.

(a) Save a 2-D array of waypoints to a binary file using `np.save()` and load it back with `np.load()`.

(b) Write the same array to a text file with `np.savetxt()` using a comma delimiter.

(c) Read the text file back with `np.loadtxt()` or `np.genfromtxt()`.

(d) List three file formats that NumPy can read or write.

Q18. Basics of Date and Time [5 marks]

Sensor logs are stamped with dates and times.

(a) Create a `datetime64` value for 2026-10-13 09:30.

(b) Create an array of five timestamps, one minute apart, using `np.arange()` with a datetime64 dtype.

(c) Calculate the time difference between the first and last timestamp.

Q19. Handling Missing Data [5 marks]

A temperature sensor failed twice: `t = np.array([21.5, np.nan, 22.1, np.nan, 23.0])`.

(a) Identify the missing values using `np.isnan()` and count them.

(b) Remove the missing values and print the cleaned array.

(c) Impute (fill) the missing values with the mean of the valid readings, using `np.nanmean()`.

(d) Why must missing values be handled before computing statistics?

Q20. Challenge: Robot Mission Log [5 marks]

A robot records distance readings (in metres) at six checkpoints, with some failures:

```
log = np.array([2.5, np.nan, 4.1, 3.8, np.nan, 5.0])
```

(a) Replace the missing values with the mean of the valid readings.

(b) Reshape the cleaned array into a 2 x 3 array and print the maximum of each row.

(c) Keep only the readings greater than 3.0 and sort them in descending order.

(d) Save the final array to a file named `mission.npy`.